

Segmentación de Clientes con Metodología RFM

Análisis de comportamiento y clasificación estratégica de clientes

Autor: RobertScience Data Consulting

Proyecto: Avance de Proyecto – Parte 3

Enfoque: Customer Analytics & Unsupervised Learning

Entorno: Python / Jupyter Notebook

Descripción del proyecto

El presente análisis tiene como objetivo segmentar clientes a partir de sus patrones de compra utilizando la metodología **RFM (Recency, Frequency, Monetary)**.

Esta técnica permite clasificar a los clientes en función de:

- **Recency (R):** Qué tan reciente fue su última compra
- **Frequency (F):** Con qué frecuencia realizan compras
- **Monetary (M):** Cuánto dinero han gastado

A partir de estos indicadores, es posible identificar distintos perfiles de clientes, tales como clientes leales, clientes en riesgo o clientes de alto valor.

Este tipo de análisis es fundamental en entornos empresariales, ya que permite optimizar estrategias de marketing, mejorar la retención de clientes y maximizar el valor del negocio.

Importación de librerías

En esta sección se importan las librerías necesarias para la manipulación de datos, análisis exploratorio y construcción del modelo de segmentación RFM.

Se utilizan herramientas estándar dentro del ecosistema de ciencia de datos en Python, lo que permite garantizar eficiencia, escalabilidad y buenas prácticas en el análisis.

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from datetime import datetime
```

Carga del conjunto de datos

En esta sección se realiza la carga del dataset que contiene información de transacciones de clientes en un entorno retail.

El dataset incluye variables como identificador de cliente, fecha de compra, cantidad de productos y valor monetario, las cuales serán fundamentales para la construcción del modelo RFM.

```
In [2]: df = pd.read_csv(  
        r"D:\Documentos\Ebac\Finales\Tarea M25-CD -RobertScience\Avance Proyecto par  
        encoding='latin-1'  
    )  
  
    df.head()
```

```
Out[2]:
```

	INVOICE_NO	STOCK_CODE	DESCRIPTION	QUANTITY	INVOICE_DATE	UNIT_PRICE
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	01/12/2019 08:26	2.55
1	536365	71053	WHITE METAL LANTERN	6	01/12/2019 08:26	3.39
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	01/12/2019 08:26	2.75
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	01/12/2019 08:26	3.39
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	01/12/2019 08:26	3.39

Exploración inicial de los datos

Antes de aplicar cualquier técnica de segmentación, es necesario analizar la estructura del dataset.

En esta etapa se revisan:

- Tipos de datos
- Valores nulos
- Comportamiento general de las variables

Este análisis permite identificar posibles problemas y preparar adecuadamente los datos para el modelo RFM.

```
In [3]: df.info()
```

```
df.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   INVOICE_NO      541909 non-null  str    
1   STOCK_CODE      541909 non-null  str    
2   DESCRIPTION      540455 non-null  str    
3   QUANTITY        541909 non-null  int64  
4   INVOICE_DATE    541909 non-null  str    
5   UNIT_PRICE      541909 non-null  float64
6   CUSTOMER_ID     406829 non-null  float64
7   REGION          541909 non-null  str    
dtypes: float64(2), int64(1), str(5)
memory usage: 33.1 MB
```

```
Out[3]:
```

	QUANTITY	UNIT_PRICE	CUSTOMER_ID
count	541909.000000	541909.000000	406829.000000
mean	9.552250	4.611114	15287.690570
std	218.081158	96.759853	1713.600303
min	-80995.000000	-11062.060000	12346.000000
25%	1.000000	1.250000	13953.000000
50%	3.000000	2.080000	15152.000000
75%	10.000000	4.130000	16791.000000
max	80995.000000	38970.000000	18287.000000

Análisis de valores nulos

Se identifican las variables que contienen valores faltantes, ya que estos pueden afectar el cálculo correcto de los indicadores RFM.

```
In [4]: df.isnull().sum()
```

```
Out[4]: INVOICE_NO      0
STOCK_CODE      0
DESCRIPTION    1454
QUANTITY        0
INVOICE_DATE    0
UNIT_PRICE      0
CUSTOMER_ID    135080
REGION          0
dtype: int64
```

Limpieza de datos

Para garantizar la calidad del análisis, se realizan los siguientes ajustes:

- Eliminación de registros sin identificador de cliente
- Eliminación de transacciones canceladas
- Eliminación de valores inválidos en cantidad y precio

Esto permite trabajar únicamente con transacciones válidas.

```
In [5]: # Eliminar clientes nulos
df = df.dropna(subset=['CUSTOMER_ID'])

# Eliminar cancelaciones (Invoice que empiezan con C)
df = df[~df['INVOICE_NO'].astype(str).str.startswith('C')]

# Eliminar valores negativos o cero
df = df[df['QUANTITY'] > 0]
df = df[df['UNIT_PRICE'] > 0]

df.head()
```

Out[5]:

	INVOICE_NO	STOCK_CODE	DESCRIPTION	QUANTITY	INVOICE_DATE	UNIT_PRICE
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	01/12/2019 08:26	2.55
1	536365	71053	WHITE METAL LANTERN	6	01/12/2019 08:26	3.39
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	01/12/2019 08:26	2.75
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	01/12/2019 08:26	3.39
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	01/12/2019 08:26	3.39

Cálculo del valor monetario

Se calcula el valor total de cada transacción multiplicando la cantidad de productos por el precio unitario.

Esta variable será utilizada para el cálculo del componente Monetary dentro del modelo RFM.

```
In [6]: df['TOTAL_PRICE'] = df['QUANTITY'] * df['UNIT_PRICE']

df.head()
```

Out[6]:

	INVOICE_NO	STOCK_CODE	DESCRIPTION	QUANTITY	INVOICE_DATE	UNIT_PRICE
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	01/12/2019 08:26	2.55
1	536365	71053	WHITE METAL LANTERN	6	01/12/2019 08:26	3.39
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	01/12/2019 08:26	2.75
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	01/12/2019 08:26	3.39
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	01/12/2019 08:26	3.39

Conversión de fechas

Se transforma la variable de fecha al formato adecuado para poder realizar cálculos relacionados con el tiempo, como la recencia de compra.

```
In [7]: df['INVOICE_DATE'] = pd.to_datetime(
    df['INVOICE_DATE'],
    dayfirst=True
)

df.info()
```

```
<class 'pandas.DataFrame'>
Index: 397884 entries, 0 to 541908
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   INVOICE_NO      397884 non-null  str
1   STOCK_CODE      397884 non-null  str
2   DESCRIPTION     397884 non-null  str
3   QUANTITY        397884 non-null  int64
4   INVOICE_DATE    397884 non-null  datetime64[us]
5   UNIT_PRICE      397884 non-null  float64
6   CUSTOMER_ID     397884 non-null  float64
7   REGION          397884 non-null  str
8   TOTAL_PRICE     397884 non-null  float64
dtypes: datetime64[us](1), float64(3), int64(1), str(4)
memory usage: 30.4 MB
```

Definición de fecha de referencia

Para calcular la recencia, es necesario definir una fecha de corte que represente el momento actual del análisis.

A partir de esta fecha se calculará cuántos días han pasado desde la última compra de cada cliente.

```
In [8]: # Fecha de referencia (última fecha del dataset + 1 día)
fecha_referencia = df['INVOICE_DATE'].max() + pd.Timedelta(days=1)

fecha_referencia
```

```
Out[8]: Timestamp('2020-12-10 12:50:00')
```

Construcción de métricas RFM

Se agrupan los datos por cliente para calcular:

- Recency: días desde la última compra
- Frequency: número de compras realizadas
- Monetary: valor total gastado

Estas métricas permiten analizar el comportamiento individual de cada cliente.

```
In [9]: rfm = df.groupby('CUSTOMER_ID').agg({
        'INVOICE_DATE': lambda x: (fecha_referencia - x.max()).days,
        'INVOICE_NO': 'nunique',
        'TOTAL_PRICE': 'sum'
    })

rfm.columns = ['Recency', 'Frequency', 'Monetary']

rfm.head()
```

```
Out[9]:
```

	Recency	Frequency	Monetary
--	---------	-----------	----------

CUSTOMER_ID			
12346.0	327	1	77183.60
12347.0	2	7	4310.00
12348.0	75	4	1797.24
12349.0	19	1	1757.55
12350.0	311	1	334.40

Análisis inicial del modelo RFM

Se analizan las métricas obtenidas para comprender la distribución del comportamiento de los clientes.

Esto permite identificar patrones como clientes frecuentes, clientes inactivos o clientes de alto valor.

```
In [10]: rfm.describe()
```

	Recency	Frequency	Monetary
count	4338.000000	4338.000000	4338.000000
mean	92.610650	4.272015	2054.266460
std	100.191727	7.697998	8989.230441
min	1.000000	1.000000	3.750000
25%	18.000000	1.000000	307.415000
50%	51.000000	2.000000	674.485000
75%	142.000000	5.000000	1661.740000
max	375.000000	209.000000	280206.020000

Asignación de puntuaciones RFM

Para facilitar la segmentación, se transforman las métricas RFM en puntuaciones.

Cada cliente recibe un score del 1 al 5 en función de su comportamiento:

- Recency: menor valor → mejor score
- Frequency: mayor valor → mejor score
- Monetary: mayor valor → mejor score

Esto permite clasificar a los clientes de forma estandarizada.

```
In [11]: # Recency (invertido: menor es mejor)
rfm['R_score'] = pd.qcut(rfm['Recency'], 5, labels=[5,4,3,2,1])

# Frequency (mayor es mejor)
rfm['F_score'] = pd.qcut(rfm['Frequency'].rank(method='first'), 5, labels=[1,2,3,4,5])

# Monetary (mayor es mejor)
rfm['M_score'] = pd.qcut(rfm['Monetary'], 5, labels=[1,2,3,4,5])

rfm.head()
```

Out[11]:

	Recency	Frequency	Monetary	R_score	F_score	M_score
CUSTOMER_ID						
12346.0	327	1	77183.60	1	1	5
12347.0	2	7	4310.00	5	5	5
12348.0	75	4	1797.24	2	4	4
12349.0	19	1	1757.55	4	1	4
12350.0	311	1	334.40	1	1	2

Construcción del score RFM

Se combinan las puntuaciones individuales en un único indicador que resume el valor del cliente.

Este score facilita la segmentación y el análisis estratégico.

In [12]:

```
rfm['RFM_score'] = (
    rfm['R_score'].astype(str) +
    rfm['F_score'].astype(str) +
    rfm['M_score'].astype(str)
)

rfm.head()
```

Out[12]:

	Recency	Frequency	Monetary	R_score	F_score	M_score	RFM_score
CUSTOMER_ID							
12346.0	327	1	77183.60	1	1	5	115
12347.0	2	7	4310.00	5	5	5	555
12348.0	75	4	1797.24	2	4	4	244
12349.0	19	1	1757.55	4	1	4	414
12350.0	311	1	334.40	1	1	2	112

Segmentación de clientes

A partir del score RFM, se clasifican los clientes en distintos segmentos estratégicos.

Esta segmentación permite identificar perfiles como:

- Clientes de alto valor
- Clientes leales
- Clientes en riesgo
- Clientes ocasionales


```
In [13]: def segmentar_cliente(row):
    if row['R_score'] >= 4 and row['F_score'] >= 4:
        return 'Cliente Premium'
    elif row['F_score'] >= 4:
        return 'Cliente Leal'
    elif row['R_score'] <= 2:
        return 'Cliente en Riesgo'
    else:
        return 'Cliente Ocasional'

rfm['Segmento'] = rfm.apply(segmentar_cliente, axis=1)

rfm.head()
```

Out[13]:

	Recency	Frequency	Monetary	R_score	F_score	M_score	RFM_score
CUSTOMER_ID							
12346.0	327	1	77183.60	1	1	5	115
12347.0	2	7	4310.00	5	5	5	555
12348.0	75	4	1797.24	2	4	4	244
12349.0	19	1	1757.55	4	1	4	414
12350.0	311	1	334.40	1	1	2	112

Distribución de segmentos

Se analiza la cantidad de clientes en cada segmento para entender la composición de la base de clientes.

```
In [14]: rfm['Segmento'].value_counts()
```

```
Out[14]: Segmento
Cliente en Riesgo    1433
Cliente Ocasional    1170
Cliente Premium      1139
Cliente Leal         596
Name: count, dtype: int64
```

Visualización de segmentos

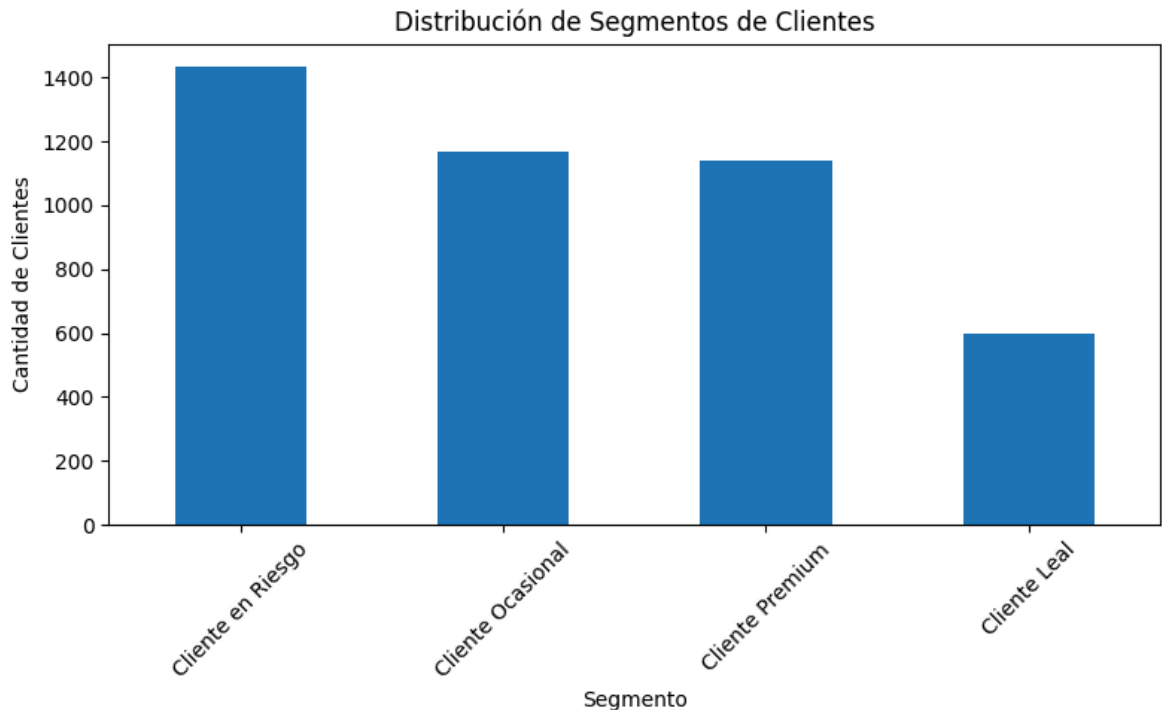
Se presenta una visualización que permite interpretar de manera clara la distribución de clientes por segmento.

```
In [15]: plt.figure(figsize=(8,5))
rfm['Segmento'].value_counts().plot(kind='bar')
```

```
plt.title('Distribución de Segmentos de Clientes')
plt.xlabel('Segmento')
plt.ylabel('Cantidad de Clientes')

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```



Insights clave

A partir del análisis realizado, se identificaron los siguientes puntos estratégicos:

- Los clientes clasificados como **Premium y Leales** representan el segmento de mayor valor, concentrando la mayor frecuencia de compra y contribución económica.
- Existe un grupo relevante de **clientes en riesgo**, caracterizados por una baja recencia, lo que indica la necesidad de implementar estrategias de reactivación.
- Los **clientes ocasionales** presentan una oportunidad de crecimiento, ya que con acciones adecuadas pueden evolucionar hacia segmentos de mayor valor.
- La distribución de segmentos evidencia una base de clientes heterogénea, lo que refuerza la importancia de aplicar estrategias diferenciadas según el perfil de cada cliente.

Conclusión del análisis

A partir del análisis realizado, se logró segmentar a los clientes utilizando la metodología RFM, lo que permitió identificar distintos patrones de comportamiento dentro de la base de datos.

El cálculo de las métricas de recencia, frecuencia y valor monetario facilitó la construcción de un modelo capaz de clasificar a los clientes en función de su actividad y contribución económica.

La asignación de puntuaciones y la posterior segmentación permitieron diferenciar perfiles estratégicos, destacando clientes de alto valor, clientes leales, así como clientes en riesgo de abandono.

Adicionalmente, la visualización de los segmentos permitió comprender la distribución de la base de clientes, proporcionando una visión clara sobre la composición del negocio.

En conjunto, este análisis demuestra cómo el uso de técnicas de segmentación permite transformar datos transaccionales en información útil para la toma de decisiones, contribuyendo a mejorar estrategias de retención, fidelización y crecimiento de clientes.

RobertScience Data Consulting

Data Science | Customer Analytics | Advanced Analytics

<https://robertscience.online/>